

ScaleGNN: Towards Scalable Graph Neural Networks via Adaptive High-order Neighboring Feature Fusion

Xiang Li
lixiang1202@stu.ouc.edu.cn
Ocean University of China
Qingdao, China

Jianpeng Qi
qijianpeng@ouc.edu.cn
Ocean University of China
Qingdao, China

Haobing Liu
haobingliu@ouc.edu.cn
Ocean University of China
Qingdao, China

Yuan Cao
cy8661@ouc.edu.cn
Ocean University of China
Qingdao, China

Guoqing Chao
guoqingchao@hit.edu.cn
Harbin Institute of Technology
Weihai, China

Zhongying Zhao
zyzhao@sdust.edu.cn
Shandong University of Science and
Technology
Qingdao, China

Junyu Dong
dongjunyu@ouc.edu.cn
Ocean University of China
Qingdao, China

Xinwang Liu
xinwangliu@nudt.edu.cn
School of Computer, National
University of Defense Technology
Changsha, China

Yanwei Yu^{*}
yuyanwei@ouc.edu.cn
Ocean University of China
Qingdao, China

Abstract

Graph Neural Networks (GNNs) have demonstrated impressive performance across diverse graph-based tasks by leveraging message passing to capture complex node relationships. However, on large-scale real-world graphs, GNNs face two major challenges: (1) GNNs struggle to ensure scalability and efficiency as repeated aggregation of large neighborhoods incurs significant computational overhead; (2) GNNs suffer from over-smoothing, where excessive propagation makes node representations indistinguishable, hindering model expressiveness. To tackle these, we propose ScaleGNN, which adaptively fuses multi-hop node features for scalable and effective graph learning. We first compute per-hop pure-neighbor matrices to isolate exclusive structural signals, then apply lightweight fusion to balance low- and high-order information, preserving both local detail and global correlations. To curb redundancy and over-smoothing, we introduce Local Contribution Score (LCS)-based masking to prune low-relevance high-order neighbors, and impose learnable sparsity to selectively integrate valuable multi-hop features. Extensive experiments on real-world datasets show that ScaleGNN consistently outperforms state-of-the-art GNNs in both predictive accuracy and computational efficiency. The source code is available at <https://github.com/lx970414/ScaleGNN>.

CCS Concepts

• **Mathematics of computing** → **Graph algorithms**; • **Computing methodologies** → **Neural networks**; **Learning latent representations**.

^{*}Yanwei Yu is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792347>

Keywords

Scalable graph neural networks, Over-smoothing, Large-scale graphs.

ACM Reference Format:

Xiang Li, Jianpeng Qi, Haobing Liu, Yuan Cao, Guoqing Chao, Zhongying Zhao, Junyu Dong, Xinwang Liu, and Yanwei Yu. 2026. ScaleGNN: Towards Scalable Graph Neural Networks via Adaptive High-order Neighboring Feature Fusion. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792347>

1 Introduction

Graph Neural Networks (GNNs) have become the cornerstone of modern graph representation learning [16, 26, 34, 46], driving advances in applications such as node classification [17], link prediction [15, 56], anomaly detection [30, 58–60], and recommender systems [29, 31, 45]. The essence of GNNs lies in iterative message-passing schemes [9, 51], whereby nodes update their representations by aggregating information from local neighborhoods [66]. This capability allows GNNs to naturally encode node representations and capture the underlying correlations between nodes.

As digital infrastructures evolve, graphs with millions or even billions of nodes and edges have become commonplace, magnifying the necessity for GNNs that are not only expressive but also computationally scalable [23, 62, 67]. In response, recent research communities have developed a rich array of scalable and deep GNN architectures, with significant innovations focused on balancing model expressiveness, computational efficiency, and training feasibility. *One line of research is sampling-based scalable GNN methods* such as GraphSAGE [14] and Cluster-GCN [4], which attempt to control computational costs by selecting subsets of neighbors during training, thereby reducing memory overhead and enabling mini-batch processing. These approaches help alleviate the burden of large-scale aggregation but may lose important structural information due to incomplete sampling. *Another stream*

is *pre-computation-based scalable approaches* (including SGC [50], SIGN [42], and GAMLP [64]), which decouple neighborhood aggregation from parameter learning, allowing feature propagation to be computed in advance and greatly accelerating both training and inference. *Meanwhile, a growing body of research has focused on deep and high-order GNNs* (e.g., S^2GC [68], GBP [2], RpHGNN [8]). They integrate information from high-order neighbors, thereby enriching the receptive field of each node and improving the capture of long-range dependencies. These advances have substantially expanded the applicability of GNNs to web-scale scenarios such as large-scale heterogeneous graphs [23, 62, 67].

Despite these substantial developments, GNNs still face two persistent and intertwined challenges when deployed on large-scale and complex graphs. *First, as the number of message-passing layers increases, GNNs suffer from the over-smoothing problem that node representations become overly similar and lose their discriminative power.* This issue is particularly severe when incorporating high-order neighbors [13, 20, 63], as their contributions may become redundant or even introduce noise, leading to performance degradation. Although several techniques, such as residual connections [27], skip connections [3, 50], and decoupling propagation [57] from feature transformation, have been proposed to alleviate over-smoothing, they often introduce additional computational complexity or fail to effectively balance local and global information. *Second, many traditional GNN architectures struggle with the scalability issue that models face high consumption and computational costs when applied to large-scale graphs* [1, 5, 28, 36, 65]. The primary reason is the exponential growth in the number of high-order neighbors, which leads to excessive information aggregation and redundant computations. Some recent works address scalability issues [8] by adopting pre-computation-based techniques, mini-batch training, or sampling strategies. While these approaches improve efficiency, they often sacrifice performance due to incomplete multi-hop neighborhood information. Recently, plug-in modules such as RMask [33] have been introduced to further enhance scalable GNNs. These modules can be seamlessly integrated into frameworks like S^2GC and GAMLP, and employ techniques such as noise masking or random walk-based neighbor selection to reduce redundancy and over-smoothing. While plug-in enhancements offer practical improvements, most still fundamentally inherit the aggregation schemes of their base models [13, 20, 63] and often lack adaptive, effective feature fusion of information from different neighborhood hops. *Overall, despite advances in sampling-based, pre-computation-based, deep and high-order aggregation, and plug-in modules, scalability and over-smoothing issues still hinder scalable GNNs for large-scale and complex graphs.*

To overcome these challenges, we propose **ScaleGNN**, a scalable framework that mitigates over-smoothing via adaptive fusion of multi-hop graph features. Specifically, we introduce a learnable mechanism to construct and refine per-hop neighbor matrices, with trainable weights adjusting the relative importance of different neighborhood orders to emphasize informative high-order neighbors while down-weighting less useful ones. A low-order-enhanced fusion further integrates low- and high-order features based on task relevance, enabling ScaleGNN to capture both local structure and global correlations with limited complexity. Furthermore, we employ a selective neighbor masking strategy, the Local Contribution

Score (LCS), and impose LCS-based sparsity to prune low-relevance high-order neighbors, which quantifies their contribution to target nodes and curbs redundancy and over-smoothing. Extensive experiments on multiple real-world graphs show that ScaleGNN consistently outperforms state-of-the-art GNNs in both accuracy and efficiency, providing a robust solution to scalability and over-smoothing in large, complex graphs.

The main contributions of our work are as follows:

- We design a trainable mechanism to construct and refine multi-hop neighbor matrices, effectively retaining informative high-order neighbors while filtering out redundancy and noise for better high-order information integration.
- We propose a low-order enhanced fusion strategy that adaptively weights low-order and high-order features, enabling the model to capture both local detail and global correlations while alleviating over-smoothing.
- We introduce the selective masking strategy named LCS to reduce multi-hop redundancy and impose sparse constraints based on LCS to prune unimportant high-order neighbors, significantly reducing computational cost with minimal performance loss.
- Experiments on real-world graphs show that our model consistently outperforms state-of-the-art GNNs both in accuracy and efficiency, demonstrating superior scalability.

2 Related Work

2.1 Graph Neural Network

GNNs learn on graphs via message passing. Representative homogeneous GNNs include GCN [22], GAT [48], GraphSAGE [14], SGC [50], APPNP [11], and S^2GC [68], which differ mainly in neighborhood aggregation and propagation strategies. Despite strong performance, these models often degrade on real-world graphs with heterophily [25, 40] or heterogeneity. We focus on the latter: by assuming homogeneous structures, these methods fail to capture type-specific semantics in heterogeneous graphs [6], where nodes and edges contribute unequally. Heterogeneous GNNs (HGNNs) are thus proposed to explicitly model such typed interactions.

Existing HGNNs can be broadly categorized into relation-wise and representation-wise approaches. Relation-wise methods aggregate neighbors along relations or meta-paths, such as HAN [49], MAGNN [10], and HetGNN [61]. Representation-wise methods perform message passing with type- or relation-aware parameters, including R-GCN [43], RSHN [69], and Transformer-based models such as HGT [18], Simple-HGN [37], and HINormer [38]. Recent work HGAMLP [32] improves scalability by adaptively fusing local and global information on large heterogeneous graphs. *Nevertheless, most HGNNs still rely on extensive message passing, leading to high training costs on large-scale graphs.*

2.2 Deep Graph Neural Networks

Deep GNNs [12, 21, 27, 41, 57] have emerged as a prevailing paradigm for scalable graph representation learning [3, 24, 63], enabling effective modeling of complex dependencies in large graphs. By stacking multiple layers, they iteratively aggregate neighborhood information to capture both local and global structures. Deep GNNs can be broadly categorized into spectral and spatial methods. Spectral approaches operate in the graph spectral domain, while spatial

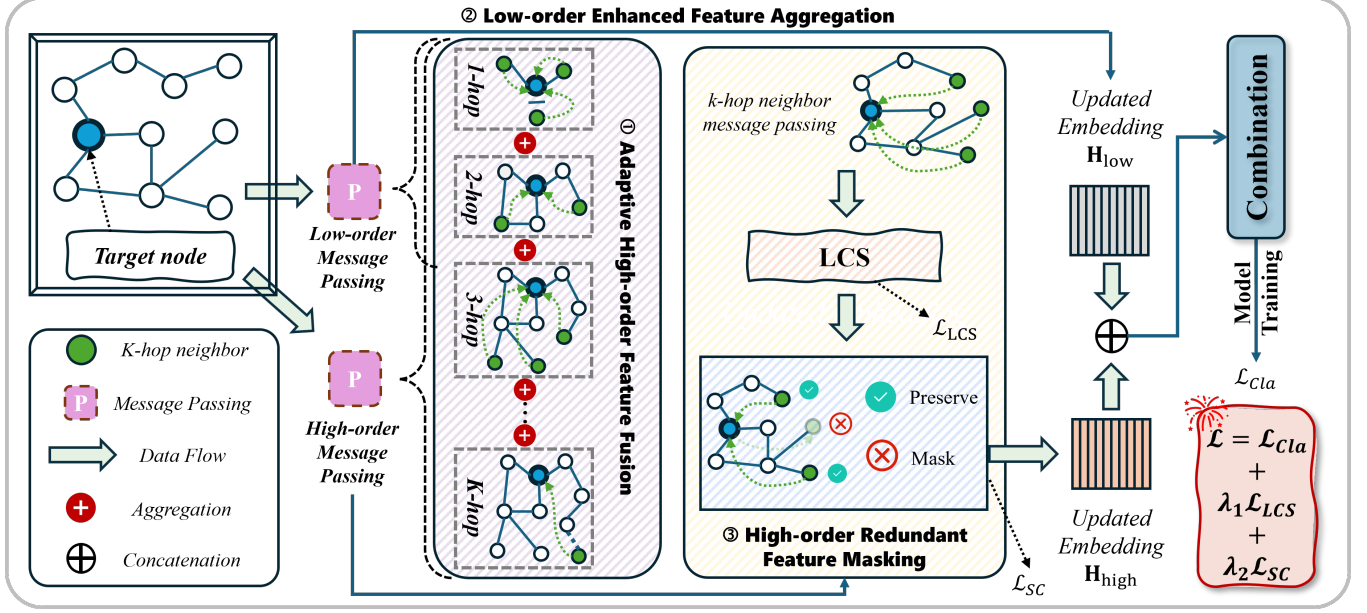


Figure 1: The overview of the proposed ScaleGNN. To address the scalability problem, ScaleGNN employs a single convolution process with a hybrid adjacency matrix that fuses features from all different neighbor hops, significantly improving computational efficiency. To mitigate the over-smoothing issue, we introduce a high-order redundant feature masking strategy, selectively filtering out less relevant high-order neighbors, which helps alleviate the over-smoothing issue.

methods aggregate directly on graph neighborhoods and are more suitable for inductive learning. Representative spatial models include GraphSAGE [14] and GAT, with later variants such as GCNII and GATv2 further improving depth, aggregation, and expressivity.

Despite these advances, increasing depth often leads to over-smoothing and rising computational cost on large-scale or dense graphs [39]. Techniques such as residual connections, identity mapping, decoupled propagation, and attention-based neighbor selection alleviate these issues but remain insufficient. *Consequently, challenges in scalability, over-smoothing, and effective utilization of high-order information persist, motivating more adaptive and efficient deep GNN frameworks.*

2.3 Scalable Graph Neural Networks

To scale GNNs on large graphs, two main paradigms have emerged: sampling-based [19, 35, 54] and pre-computation-based methods [17, 53]. Sampling reduces computational cost by selecting subsets of neighbors but may introduce variance and information loss. In contrast, pre-computation decouples propagation from nonlinear transformation, enabling parameter-free diffusion to be computed offline and yielding more stable and efficient training. SGC [50] exemplifies this idea by removing inter-layer nonlinearities, while S²GC [68] and related works extend expressiveness via multi-hop propagation and adaptive fusion without sacrificing scalability.

Extending pre-computation to heterogeneous graphs is more challenging due to multiple node and edge types. NARS [55] samples relation subsets and performs relation-specific pre-computation to reduce information loss. SeHGNN [53] aggregates relation-wise features within K hops, preserving fine-grained signals but incurring high cost as the number of relations or K increases. RpHGNN [17]

addresses this issue with a hybrid design that balances relation-wise fidelity and representation-wise efficiency.

Recent plug-in modules, such as RMask [33], further enhance scalable GNNs by filtering propagation noise, mitigating over-smoothing, and reducing pre-processing overhead, enabling deeper diffusion and more effective feature aggregation on large graphs.

In summary, scalable GNNs improve efficiency through sampling, pre-computation, and plug-in designs, while relation-aware and adaptive aggregation are crucial. *Nevertheless, balancing scalability and information preservation, reducing sampling variance, and alleviating over-smoothing remain open challenges.*

3 Preliminary

Generally, we consider an undirected large-scale graph as $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathcal{X}\}$, where $\mathcal{V}$ is the collection of nodes, $\mathcal{E}$ is the collection of edges between nodes, and $\mathcal{X} \in \mathbb{R}^{|\mathcal{V}| \times f}$ is the collection of node attributes, where f is the size of the node feature vector. $\mathbf{A} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ denotes the adjacency matrix of graph $\mathcal{G}$. $\mathbf{D} = \text{diag}(d_1, d_2, \dots, d_{|\mathcal{V}|}) \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ represents the degree matrix of $\mathbf{A}$, where $d_i = \sum_{v_j \in \mathcal{V}} A_{ij}$ denotes the degree of node v_i . Given the labeled node set $\mathcal{V}_l$, our goal is to predict the class labels for nodes in the unlabeled set $\mathcal{V}_u = \mathcal{V} - \mathcal{V}_l$ under the supervision of $\mathcal{V}_l$.

4 Methodology

In this section, we present ScaleGNN, an efficient end-to-end framework for scalable and robust GNN learning. ScaleGNN addresses over-smoothing, computational inefficiency, and redundant high-order information through three synergistic components: (1) *Adaptive High-order Feature Fusion*, (2) *Low-order Enhanced Feature Aggregation*, and (3) *High-order Redundant Feature Masking*. These

modules enable ScaleGNN to process large-scale graphs efficiently while preserving essential information across neighborhood hops. The overall framework is illustrated in Fig. 1.

4.1 Adaptive High-order Feature Fusion

Traditional GNNs are based on message passing through fixed adjacency matrices, where the adjacency structure remains constant throughout the layers. This mechanism can lead to over-smoothing, where node representations converge and lose their distinctiveness as the network depth increases. Furthermore, as nodes aggregate information from higher-order neighbors, the feature similarity across nodes can become excessive, thereby reducing the model's expressiveness.

To mitigate these issues, we propose a method for constructing distinct adjacency matrices for each neighborhood hop. This allows us to isolate the relations specific to each hop, ensuring that each neighborhood level contributes uniquely to the node representation. We define the *pure* i -th hop adjacency matrix A_i as the difference between the $(i-1)$ -th and i -th hops of adjacency matrices:

$$A_i = A^i - A^{i-1}, \quad i = 1, 2, 3, \dots, K, \quad (1)$$

where $A^i \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ is the i -th hop adjacency matrix, which captures the connections up to i hops, A_i isolates the new relations emerging specifically at the i -th hop. K is the maximum number of hops, also referred to as the order. Specifically, both A_1 and A^1 are the adjacency matrix of the graph, and A^0 is the zero matrix.

Incorporating information from multiple hops can provide a more comprehensive view of the graph structure. However, the challenge lies in integrating these multiple hops in a way that preserves their distinct contributions. To address this, we introduce a learnable weight set $\alpha = [\alpha_1, \alpha_2, \dots, \alpha_K]$, where each $\alpha_i \in \mathbb{R}$ ($i \in \{1, 2, \dots, K\}$) represents the specific important variant of the i -th hop adjacency matrix in the final aggregation:

$$\tilde{A} = \sum_{i=1}^K \alpha_i A_i, \quad \sum_{i=1}^K \alpha_i = 1. \quad (2)$$

These weights are learned through back-propagation, ensuring that the model dynamically adjusts the contribution of each neighborhood hop. The use of the softmax function to optimize these weights ensures that contributions across different hops remain balanced. Finally, we obtain the adaptive high-order fused embedding matrix $H \in \mathbb{R}^{|\mathcal{V}| \times d_f}$ as follows:

$$H = \sigma(\tilde{A} \cdot \mathcal{X} \cdot W), \quad (3)$$

where d_f is the hidden embedding dimension, $\mathcal{X} \in \mathbb{R}^{|\mathcal{V}| \times f}$ is the input feature matrix, and $W \in \mathbb{R}^{f \times d_f}$ is the learnable weight matrix for high-order features. The activation function $\sigma(\cdot)$ introduces non-linearity to the feature transformation.

Specifically, we use the current model as our base version named **ScaleGNN_b**, which is relatively simple and efficient.

4.2 Low-order Enhanced Feature Aggregation

The integration of low-order (local) and high-order (global) features in many GNN-based models is done in a single and often static step. However, this can lead to high-order features dominating over low-order ones, which might diminish the model's ability

to preserve fine-grained local information. To address this, we propose explicitly separating low-order and high-order features before fusion. First, we obtain the low-order feature matrix H_{low} and the high-order feature matrix H_{high} as follows:

$$H_{\text{low}} = \sigma(A^2 \cdot \mathcal{X} \cdot W_{\text{low}}), \quad H_{\text{high}} = \sigma(\tilde{A} \cdot \mathcal{X} \cdot W_{\text{high}}), \quad (4)$$

where $W_{\text{low}}, W_{\text{high}} \in \mathbb{R}^{f \times d}$ are both learnable weight matrices for low-order and high-order features, respectively.

To ensure a balanced integration of low-order and high-order features, we introduce a learnable balancing factor β , which allows for dynamic adjustment of diverse contributions from two feature types. The final node representation H is the weighted sum of low-order and high-order features:

$$H = \beta \cdot H_{\text{low}} + (1 - \beta) \cdot H_{\text{high}}, \quad (5)$$

where β is the hyperparameter, ensuring the model can effectively balance the trade-off between local and global information, depending on the specific task and graph structure.

This separation and fusion process enhances the model's ability to capture both fine-grained, local details and broader, global patterns in the graph, ultimately leading to more expressive and robust node embeddings.

4.3 High-order Redundant Feature Masking

High-order neighbors introduce diverse, yet potentially redundant, information to GNNs. These neighbors will introduce noise without an effective masking mechanism, diluting the informative signal and impairing the model's ability to generate useful node embeddings. To address this, we propose a novel method for selecting the most relevant and masking other redundant high-order neighbors based on their local contributions to node representations.

Inspired by the scaled dot-product attention mechanism [47], we introduce the Local Contribution Score (LCS) to assign importance scores to high-order neighbors. The LCS is designed to quantify the relevance of each neighboring node of the target node by considering both structural similarity and feature alignment. Specifically, for a node v and its i -th hop neighbor u , the LCS value is:

$$\text{LCS}(v, u, i) = \frac{\exp\left(\frac{(W_1 x_v)^\top (W_2 x_u)}{\sqrt{d_f}}\right)}{\sum_{u' \in \mathcal{N}_i(v)} \exp\left(\frac{(W_1 x_v)^\top (W_2 x_{u'})}{\sqrt{d_f}}\right)}. \quad (6)$$

Here, $x_v, x_u \in \mathbb{R}^f$ are the input feature vectors of the target node v and its k -hop neighbor u . $W_1, W_2 \in \mathbb{R}^{d_f \times f}$ are learnable projection matrices that map features into a latent attention space of dimension d_f , which is a tunable hyperparameter, and typically $d_f \leq f$ to ensure efficiency. $\mathcal{N}_i(v)$ denotes the i -hop neighborhood of node v induced by A_i . The denominator normalizes attention scores within each local i -hop neighborhood using the softmax function. This equation allows the model to learn context-aware neighbor importance scores in a differentiable and robust way.

The LCS allows us to mask redundant high-order neighbors and identify the most relevant neighbors by considering both local structure and feature similarity, enabling the model to selectively pass information from the most meaningful high-order neighbors. Once the LCS is computed for each neighbor, we select the top- m_i

Table 1: Statistical summaries of datasets.

Datasets	#Nodes	#Edges	#Features	#Classes	#Train/Val/Test	Description
Citeseer	3,327	4,732	3,703	6	120/500/1000	citation network
Cora	2,708	5,429	1,433	7	140/500/1000	citation network
Pubmed	19,717	44,338	500	3	60/500/1000	citation network
ogbn-arxiv	169,343	1,166,243	128	40	91k/30k/49k	citation network
ogbn-products	2,449,029	61,859,140	100	47	196k/49k/2,204k	co-purchasing network
ogbn-papers100M	111,059,956	1,615,685,872	128	172	1200k/200k/146k	citation network

neighbors with the highest LCS values for each node among its i -hop neighbors:

$$\mathbf{A}_i^{\text{filter}} = \text{Top}_{m_i}(\mathbf{A}_i, \text{LCS}), \quad (7)$$

where $\text{Top}_{m_i}(\cdot, \cdot)$ retains the m_i entries with highest scores for each row in $\mathbf{A}_i$. Each m_i specifies the number of neighbors to be sampled from the i -hop neighborhood of nodes. Notice that in our framework, m_i ($i \in \{1, 2, \dots, K\}$) is a learnable parameter. By learning distinct m_i values, the model adaptively controls the neighbor sampling count at different hops, facilitating flexible and hierarchical aggregation of neighborhood information. This mechanism ensures that only the most informative high-order neighbors contribute to the final node representation, thereby effectively mitigating noise and enhancing embedding stability.

4.4 Joint Optimization

To ensure the effectiveness of ScaleGNN, we design an optimization framework that consists of three key loss components: the node classification task loss, the LCS regularization loss, and the sparse constraint loss on the high-order information. These losses work together to enhance model performance, suppress noise from uninformative high-order neighbors, and maintain computational efficiency. For node classification, we employ the standard cross-entropy loss as the primary task objective:

$$\mathcal{L}_{\text{Cla}} = - \sum_{i \in \mathcal{V}} y_i \log \hat{y}_i, \quad (8)$$

where y_i represents the one-hot vector of the ground-truth class label for node i , and $\hat{y}_i$ is the predicted probability distribution over classes generated by a softmax layer.

To further mitigate the over-smoothing issue, we impose a sparse constraint (SC) on high-order feature propagation:

$$\mathcal{L}_{\text{SC}} = \sum_{i=2}^K \|\bar{\mathbf{A}}_i^{\text{filter}}\|_1, \quad (9)$$

where $\bar{\mathbf{A}}_i^{\text{filter}}$ is the filtered high-order adjacency matrix, $\bar{\mathbf{A}}_i^{\text{filter}} = \mathbb{I}[\mathbf{A}_i^{\text{filter}} > 0]$, where $\mathbb{I}[\cdot]$ denotes the element-wise indicator function, and $\|\cdot\|_1$ denotes the $\mathcal{L}_1$ normalization. This term constrains the model to minimize the number of neighbors retained for each node at each hop.

To guide the model in selecting meaningful high-order neighbors, we also introduce an LCS-based regularization:

$$\mathcal{L}_{\text{LCS}} = \sum_{v \in \mathcal{V}} \sum_{i=2}^K \sum_{u \in \mathcal{N}_i^{\text{select}}(v)} (1 - \text{LCS}(v, u, i))^2, \quad (10)$$

where $\mathcal{N}_i^{\text{select}}(v)$ is the retained neighboring nodes which are not been filtered. This term enforces high-order neighbors to be selected based on their LCS values, reducing the influence of noisy or redundant neighbors. The final optimization objective is as follows:

$$\mathcal{L} = \mathcal{L}_{\text{Cla}} + \lambda_1 \mathcal{L}_{\text{LCS}} + \lambda_2 \mathcal{L}_{\text{SC}}, \quad (11)$$

where λ_1 and λ_2 are hyperparameters balancing regularization against the primary task objective. Specifically, enforcing sparsity constraints during the selection of higher-order neighbors per hop allows the model to strike a balance between preserving critical features and mitigating redundant information.

Overall, this optimization framework ensures that our ScaleGNN maintains a balance between node classification performance, robust high-order neighbor aggregation, and computational efficiency. The algorithm of our ScaleGNN and time complexity analysis are in the Appendix sections A.1 and A.2, respectively.

5 Experiments

In this section, we evaluate the performance of our proposed method through extensive experiments and answer the following questions:

- **(RQ1)** Can ScaleGNN outperform SOTA GNN methods regarding predictive accuracy on real-world datasets?
- **(RQ2)** How does the efficiency of ScaleGNN compare to other baseline methods?
- **(RQ3)** How does ScaleGNN mitigate the over-smoothing issue?
- **(RQ4)** What are the effects of different modules in ScaleGNN?
- **(RQ5)** How do different hyperparameter settings affect ScaleGNN?
- **(RQ6)** How does ScaleGNN achieve the trade-off between efficiency and accuracy?

5.1 Experimental Settings

5.1.1 Datasets. We have evaluated the effectiveness of ScaleGNN using six real-world graph datasets, three small-scale datasets Citeseer, Cora, and Pubmed, and three large-scale datasets ogbn-arxiv, ogbn-products, and ogbn-papers100M. The detailed introduction of the used datasets is in the Appendix section A.3.

5.1.2 Baselines. We compare our method with three categories of GNN baselines as follows: (1) traditional GNNs: R-GCN [43], Cluster-GCN [4], HetGNN [61], HAN [49], MAGNN [10], Simple-HGN [37], and HINormer [38]; (2) deep GNNs: SIGN [42], S²GC [68], GBP [2], GAMLP [64], GRAND+ [7], and LazyGNN [52]; (3) scaleable GNNs: NARS [55], SeHGNN [53], RpHGNN [17], and TOP [44]. The detailed description of all baselines is in the Appendix section A.4.

5.1.3 Implementation Details. The detailed implementation of our ScaleGNN can be found in the Appendix section A.5.

Table 2: Performance comparison of all models on six datasets. Mi-F1 and Ma-F1 are short for Micro-F1 and Macro-F1. Marker * indicates the results are statistically significant against the best-performed baselines (t-test with p-value < 0.01).

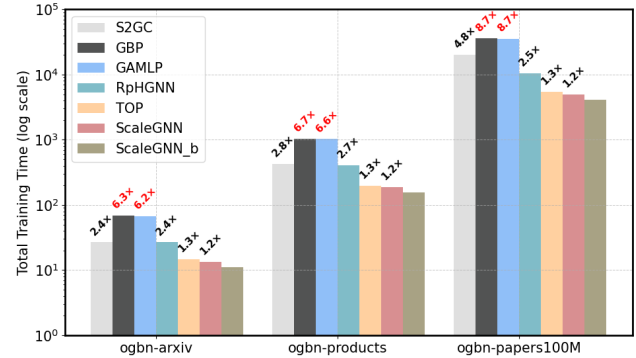
Method		Citeseer		Cora		Pubmed		ogbn-arxiv		ogbn-products		ogbn-paper100M	
		Mi-F1	Ma-F1	Mi-F1	Ma-F1	Mi-F1	Ma-F1	Mi-F1	Ma-F1	Mi-F1	Ma-F1	Mi-F1	Ma-F1
Trad. GNN	RGCN	70.53	70.14	81.23	80.59	78.37	75.58	58.26	46.83	48.94	48.04	42.31	31.12
	Cluster-GCN	71.25	70.72	82.10	80.88	78.84	76.06	78.96	75.95	71.59	63.88	62.34	54.02
	HetGNN	70.55	70.13	81.20	80.57	78.40	75.66	OOM.	OOM.	OOM.	OOM.	OOM.	OOM.
	HAN	71.10	70.57	81.39	80.86	78.53	75.85	54.68	31.50	OOM.	OOM.	OOM.	OOM.
	MAGNN	71.22	70.69	81.57	80.98	78.55	76.01	OOM.	OOM.	OOM.	OOM.	OOM.	OOM.
	Simple-HGN	71.34	70.78	81.88	81.14	78.88	76.17	69.47	58.29	OOM.	OOM.	OOM.	OOM.
	HINormer	72.75	71.91	82.19	81.67	79.35	76.60	70.71	59.68	OOM.	OOM.	OOM.	OOM.
Deep GNN	SIGN	72.44	71.97	82.11	80.46	79.53	76.44	71.78	62.99	78.21	70.10	62.32	53.59
	S ² GC	72.78	72.13	82.45	82.09	79.56	76.67	71.88	63.23	77.09	69.54	62.63	53.80
	GBP	72.61	72.08	83.42	<u>82.42</u>	80.56	79.13	71.42	62.41	77.27	69.49	62.66	53.97
	GAMLP	72.51	72.06	82.31	<u>80.32</u>	79.11	76.15	71.88	63.11	80.34	71.34	62.53	53.79
	GRAND+	72.48	72.03	82.04	80.23	79.24	76.46	71.43	62.30	77.52	69.70	61.85	53.49
	LazyGNN	72.91	72.21	82.45	80.87	79.92	77.03	71.63	62.72	77.60	69.90	61.97	53.55
Scal. GNN	NARS	72.41	71.21	81.79	80.30	78.82	75.82	70.75	59.88	78.08	70.20	62.52	53.63
	SeHGNN	72.90	72.45	83.04	81.43	79.71	77.70	71.82	62.69	79.68	70.85	63.75	54.74
	RpHGNN	<u>73.12</u>	72.54	83.27	82.37	80.58	79.32	<u>72.06</u>	63.34	80.89	71.44	63.95	55.50
	TOP	73.11	<u>72.69</u>	83.30	82.35	<u>80.60</u>	<u>79.35</u>	71.82	62.90	<u>80.93</u>	<u>71.45</u>	63.95	55.48
Ours	ScaleGNN _b	72.95	72.47	<u>83.42</u>	82.40	80.48	79.15	71.68	62.65	79.75	70.87	<u>63.97</u>	<u>55.60</u>
	ScaleGNN	74.44*	73.76*	84.75*	84.21*	82.11*	80.67*	73.25*	64.20*	81.82*	72.47*	65.69*	57.42*

5.2 Performance Comparison (RQ1)

We evaluate the performance of our ScaleGNN and all baselines on six real-world datasets under the node classification task. The experimental results are shown in Table 2. The best results are highlighted in bold, and the second-best results are underlined. As we can see, our ScaleGNN achieves the optimal performance, significantly outperforming all baselines in both micro-F1 and macro-F1 metrics on six datasets, with specific emphasis on the improvement relative to the SOTA methods.

Traditional GNN methods work flawlessly on small-scale datasets, but most of them cannot be extended to large-scale graphs and are prone to OOM issues. For example, R-GCN, HAN, Simple-HGN, and HINormer can get experimental results on ogbn-arxiv, but cannot run on larger-scale graphs such as ogbn-products and ogbn-paper100M. For deep GNN methods, they work on learning large-scale graph representations using deeper GNNs while mitigating the over-smoothing problem on large-scale graphs. A common issue with such methods is that they typically convolve dozens or even tens of GNN layers. While capturing the caveat features to some extent, it also greatly increases the computational complexity of the model. Pre-computation-based GNNs are distinguished from end-to-end models by the fact that they consume a significant amount of time in pre-computing the tensor of large-scale graphs, so that the models do not need to iteratively learn graph representations during training and inference.

It is worth mentioning that our basic model ScaleGNN_b, although retaining only the basic adaptive high-order feature fusion module, achieves a very similar performance to SOTA methods and runs efficiently, far outperforming all baselines. ScaleGNN gains further performance despite being less efficient than ScaleGNN_b. Therefore,

**Figure 2: Efficiency comparison with SOTA GNN methods, and speedup analysis compared with our ScaleGNN_b.**

ScaleGNN and ScaleGNN_b provide an important reference value for us to explore the trade-off between model accuracy and efficiency.

5.3 Efficiency Analysis (RQ2)

To evaluate the efficiency of our proposed methods, ScaleGNN and ScaleGNN_b, we compare their total runtime with several state-of-the-art baselines, including S²GC, GBP, GAMLP, and RpHGNN, across three benchmark datasets: ogbn-arxiv, ogbn-products, and ogbn-papers100M. The results presented in Fig. 2 clearly demonstrate the superior efficiency of ScaleGNN and ScaleGNN_b over all competing methods.

5.3.1 Computation Efficiency. As shown in the figure, ScaleGNN_b achieves the lowest runtime across all datasets, followed closely by ScaleGNN. Specifically, on the large-scale ogbn-papers100M

dataset, ScaleGNN_b completes the model running in only 4,110.2 seconds, significantly outperforming S²GC (19,914.4s), GBP (35,706.1s), GAMLP (35,656.6s), and RpHGNN (10,399.8s). This trend is also consistently observed across ogbn-arxiv and ogbn-products datasets, where ScaleGNN_b reduces runtime by at least 4.1× to 6.7× compared to the best-performing baseline. The significant reduction in runtime can be attributed to the efficient design of ScaleGNN, which optimizes neighborhood aggregation and selectively samples informative neighbors, thereby reducing computational overhead. Moreover, the enhanced scalability of ScaleGNN_b further refines this efficiency by introducing additional optimizations in feature propagation and memory management.

5.3.2 Contrastive Speedup Analysis. To better illustrate the efficiency advantage, we annotate each baseline method with its relative speedup factor compared to ScaleGNN_b. It is evident that methods like GBP and GAMLP exhibit substantially higher computational costs, with runtime exceeding 6× that of ScaleGNN_b in certain cases. Even RpHGNN, which demonstrates competitive efficiency, remains 2.5× to 3.1× slower than ScaleGNN_b on large datasets. These results highlight the capability of ScaleGNN to effectively mitigate the scalability limitations of existing methods. By reducing redundant computations and dynamically adjusting the information flow, our approach enables substantial efficiency improvements without compromising accuracy. The consistent speedup across diverse datasets further underscores the robustness of our design, making it particularly suitable for large-scale graph representation learning tasks.

In summary, ScaleGNN and ScaleGNN_b achieve remarkable computational efficiency, substantially reducing runtime compared to existing approaches. The combination of selective neighbor aggregation, adaptive feature diffusion, and memory-efficient propagation mechanisms enables our methods to handle large-scale graphs with minimal computational overhead. These findings underscore the practical applicability of ScaleGNN in real-world scenarios where the model efficiency is a critical concern.

5.4 Mitigating Over-smoothing Issue (RQ3)

Over-smoothing is a well-known issue in deep graph neural networks, where node representations become indistinguishable as the number of propagation layers increases. This problem is particularly severe in large-scale graphs such as ogbn-papers100M, where excessive feature mixing leads to performance degradation. To evaluate the effectiveness of our approach in addressing this challenge, we compare ScaleGNN with several SOTA baselines, including S²GC, GBP, GAMLP, and RpHGNN, across different propagation depths.

Fig. 3 shows that traditional GNNs experience significant performance drops as the number of propagation layers increases. For example, S²GC, GBP, and GAMLP achieve their peak performance at moderate depths (around 8–12 layers) but degrade rapidly beyond this point. This phenomenon occurs because deeper layers cause excessive aggregation, leading to the loss of discriminative node features. In contrast, ScaleGNN exhibits a more stable performance curve. Even at greater depths (e.g., 30 layers), ScaleGNN maintains a high micro-F1 score, outperforming all baselines, which demonstrates that ScaleGNN effectively preserves structural and feature information, preventing the collapse of node representations.

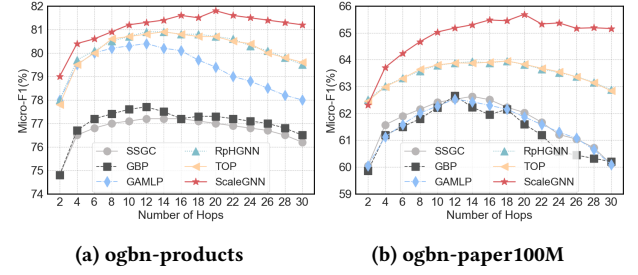


Figure 3: Experimental results of our model over SOTA GNN models w.r.t. the number of hops.

The key to ScaleGNN’s resilience against over-smoothing lies in its adaptive feature aggregation mechanism. Unlike conventional approaches that uniformly aggregate neighborhood information, ScaleGNN dynamically balances local and global features, ensuring that useful information is retained while mitigating excessive smoothing. Specifically, our ScaleGNN selectively incorporates higher-order neighbors with controlled influence, preventing over-mixing of features. It employs adaptive diffusion mechanisms to regulate information propagation, ensuring a balance between short-term and long-term dependencies. ScaleGNN utilizes LCS to prioritize informative neighbors, reducing noise and redundancy in feature aggregation. These strategies allow ScaleGNN to maintain a richer set of node representations, even in deep architectures, making it particularly effective for large-scale graphs.

5.5 Ablation Study (RQ4)

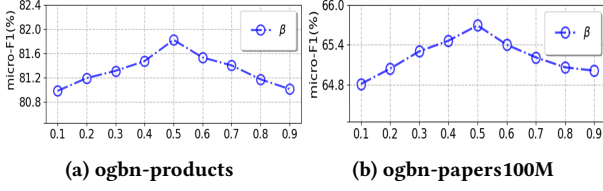
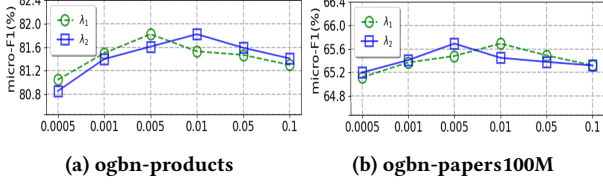
To evaluate the effectiveness of each component in ScaleGNN, we further conduct ablation studies on different variants. Specifically, we generate three variants as follows:

- **w/o Ada** removes the adaptive high-order feature fusion module and employs the deep GNN method S²GC [68] to obtain the graph representation.
- **w/o Low** excludes the low-order enhanced feature aggregation and only employs adaptive high-order feature fusion.
- **w/o LCS** removes the high-order redundant feature masking.
- **w/o SC** removes the masking sparse constraint and keeps the same retained number of nodes in each A_k^{filter} .

Table 3: The comparison of ScaleGNN and its variants in terms of Mi-F1 and Ma-F1 on three large-scale datasets.

Dataset Metrics	ogbn-arxiv		ogbn-products		ogbn-paper100M	
	Mi-F1	Ma-F1	Mi-F1	Ma-F1	Mi-F1	Ma-F1
w/o Ada	70.13	62.03	79.82	70.31	64.19	55.87
w/o Low	71.92	62.85	79.97	71.22	64.97	56.56
w/o LCS	71.74	62.17	80.08	70.83	64.45	55.98
w/o SC	72.52	63.87	81.26	71.77	65.46	57.19
ScaleGNN	73.25	64.20	81.82	72.47	65.69	57.42

The results demonstrate that each component is vital for improving ScaleGNN’s performance. **w/o Ada** variant experiences the largest performance drop across all datasets, with reductions of 3.07% in Micro-F1 and 2.17% in Macro-F1 on ogbn-arxiv, and 1.14% on ogbn-products, and 1.24% and 1.44% on ogbn-paper100M,

Figure 4: The effect of balanced hyperparameter β .Figure 5: The effect of loss function weights λ_1 and λ_2 .

highlighting the importance of adaptive high-order feature fusion for capturing structural dependencies and mitigating information loss. **w/o Low** variant also shows notable degradation, especially on ogbn-arxiv and ogbn-paper100M, with Micro-F1 dropping by 1.33% and 0.72%, respectively, indicating that low-order enhanced feature aggregation is crucial for maintaining local information. **w/o LCS** variant sees moderate degradation, with Micro-F1 decreasing over three large-scale datasets, showing the effectiveness of high-order redundant feature masking in reducing redundancy and preventing excessive feature mixing. **w/o SC** shows that it still falls short of the full-fledged ScaleGNN model despite being the best of all variants, demonstrating the important role of the LCS-based sparse constraints. It employs an attention-based relevance function, which allows ScaleGNN to adaptively assess and filter neighbors in a differentiable and task-driven manner, significantly improving its generalization and scalability in large-scale graphs.

Overall, ScaleGNN consistently outperforms all variants, confirming the significant contribution of each component. Adaptive high-order feature fusion is the most influential in improving performance, followed by low-order enhanced feature aggregation and high-order redundant feature masking. The combination of these mechanisms enables ScaleGNN to achieve superior results on various large-scale graph datasets.

5.6 Parameter Sensitivity Analysis (RQ5)

5.6.1 The effect of hyperparameters β . We conduct a sensitivity analysis of β , which balances low-order and high-order feature aggregations, by varying its value from 0.1 to 0.9 and evaluating the Micro-F1 score (%). The results in Fig. 5 show that performance improves as β increases, peaking at $\beta = 0.5$. Beyond this point, further increases in β lead to a gradual decline, likely due to overemphasizing high-order information, which may introduce noise or redundancy. These findings highlight the importance of β to strike an optimal balance between local and global information aggregation.

5.6.2 The effect of hyperparameters λ_1 and λ_2 . We assess the sensitivity of λ_1 and λ_2 in the loss function by varying their values within $\{0.0005, 0.001, 0.005, 0.01, 0.05, 0.1\}$ and evaluating Micro-F1. As shown in Fig. 5, ScaleGNN achieves the best performance at $\lambda_1 = 0.005$, with results improving up to this point and declining thereafter, reflecting the benefit of moderate regularization.

In contrast, performance with respect to λ_2 is more stable, peaking at $\lambda_2 = 0.01$ with only slight variations, indicating its role in maintaining model stability. These results highlight the need for careful tuning: λ_1 requires precise adjustment for optimal results, while λ_2 ensures robustness. The loss terms weighted by λ_1 and λ_2 complement each other, balancing performance and efficiency.

5.7 Trade-off Analysis: Performance vs. Efficiency (RQ6)

In this section, we evaluate the trade-off between efficiency and accuracy for ScaleGNN and ScaleGNN_b by analyzing their runtime and Micro-F1 scores across diverse hop settings.

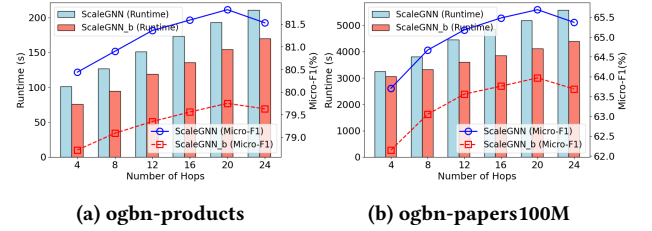


Figure 6: The trade-off between efficiency and accuracy on large-scale graph datasets.

The runtime analysis reveals in Fig. 6 that ScaleGNN incurs a higher computational cost than ScaleGNN_b across all hop settings. As the number of hops increases, both methods show rising runtimes, reflecting the increased complexity of message passing over larger neighborhoods. However, ScaleGNN consistently requires more computational resources, suggesting that its modeling choices introduce additional overhead. ScaleGNN consistently outperforms ScaleGNN_b in Micro-F1 scores. Both methods benefit from increasing hops, as it helps capture structural and semantic relationships. However, beyond a certain point, performance gains diminish due to redundant or noisy information from distant nodes. These results highlight the trade-off between accuracy and efficiency. While ScaleGNN achieves higher Micro-F1 scores, it comes with a significantly higher computational cost. In contrast, ScaleGNN_b offers a more efficient alternative with a slight performance compromise. This suggests that model selection should be based on practical constraints: ScaleGNN is ideal for accuracy-focused tasks, while ScaleGNN_b is better for resource-constrained environments.

6 Conclusion

We propose a novel scalable GNN model named ScaleGNN, tackling over-smoothing and scalability challenges in large-scale graphs. ScaleGNN adaptively aggregates informative high-order neighbors while suppressing redundancy through a Local Contribution Score (LCS)-based masking mechanism. Extensive experiments show that ScaleGNN outperforms state-of-the-art deep and scalable GNNs in both accuracy and efficiency. Future work includes extending it to heterogeneous graphs and optimizing large-scale training.

Acknowledgments

This work is supported by the Advanced Materials-National Science and Technology Major Project (Grant No. 2024ZD0607700).

References

- [1] Hao Chen, Yuanchen Bei, Qijie Shen, Yue Xu, Sheng Zhou, Wenbing Huang, Feiran Huang, Senzhang Wang, and Xiao Huang. 2024. Macro graph neural networks for online billion-scale recommender systems. In *WWW*. 3598–3608.
- [2] Ming Chen, Zhewei Wei, Bolin Ding, Yaliang Li, Ye Yuan, Xiaoyong Du, and Ji-Rong Wen. 2020. Scalable graph neural networks via bidirectional propagation. *NeurIPS* 33 (2020), 14556–14566.
- [3] Tianlong Chen, Kaixiong Zhou, Keyu Duan, Wenqing Zheng, Peihao Wang, Xia Hu, and Zhangyang Wang. 2022. Bag of tricks for training deeper graph neural networks: A comprehensive benchmark study. *IEEE TPAMI* 45, 3 (2022), 2769–2781.
- [4] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. 2019. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. In *KDD*. 257–266.
- [5] Mucong Ding, Tahseen Rabbani, Bang An, Evan Wang, and Furong Huang. 2022. Sketch-GNN: Scalable graph neural networks with sublinear training complexity. *NeurIPS* 35 (2022), 2930–2943.
- [6] Wei Du, Hongmin Sun, Hang Gao, Gaoyang Li, and Ying Li. 2025. Contrastive Auxiliary Learning with Structure Transformation for Heterogeneous Graphs. In *AAAI*, Vol. 39. 16399–16407.
- [7] Wenzheng Feng, Yuxiao Dong, Tinglin Huang, Ziqi Yin, Xu Cheng, Evgeny Kharlamov, and Jie Tang. 2022. Grand+: Scalable graph random neural networks. In *WWW*. 3248–3258.
- [8] Ben Finkelshtein, Ismail Ceylan, Michael Bronstein, and Ron Levie. 2024. Learning on large graphs using intersecting communities. *NeurIPS* 37 (2024), 57349–57388.
- [9] Chaofan Fu, Guanjie Zheng, Chao Huang, Yanwei Yu, and Junyu Dong. 2023. Multiplex heterogeneous graph neural network with behavior pattern modeling. In *KDD*. 482–494.
- [10] Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. 2020. Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding. In *WWW*. 2331–2341.
- [11] Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Predict then propagate: Graph neural networks meet personalized pagerank. *arXiv* (2018).
- [12] Jhony H Giraldo, Konstantinos Skianis, Thierry Bouwmans, and Fragkiskos D Malliaros. 2023. On the trade-off between over-smoothing and over-squashing in deep graph neural networks. In *CIKM*. 566–576.
- [13] Kaiqi Gong, Xiao Song, Wenxin Li, and Senzhang Wang. 2024. HN-GCCF: High-order neighbor-enhanced graph convolutional collaborative filtering. *Knowledge-Based Systems* 283 (2024), 111122.
- [14] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *NeurIPS* 30 (2017).
- [15] Jinquan Hang, Zhiqing Hong, Xinyue Feng, Guang Wang, Guang Yang, Feng Li, Xining Song, and Desheng Zhang. 2024. Paths2pair: Meta-path based link prediction in billion-scale commercial heterogeneous graphs. In *KDD*. 5082–5092.
- [16] Zhenyu Hou, Yufei He, Yukuo Cen, Xiao Liu, Yuxiao Dong, Evgeny Kharlamov, and Jie Tang. 2023. Graphmae2: A decoding-enhanced masked self-supervised graph learner. In *WWW*. 737–746.
- [17] Jun Hu, Bryan Hooi, and Bingsheng He. 2024. Efficient heterogeneous graph learning via random projection. *IEEE TKDE* (2024).
- [18] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous graph transformer. In *Proceedings of the Web Conference 2020*. 2704–2710.
- [19] Xunqiang Jiang, Tianrui Jia, Yuan Fang, Chuan Shi, Zhe Lin, and Hui Wang. 2021. Pre-training on large-scale heterogeneous graph. In *KDD*. 756–766.
- [20] Di Jin, Yingli Gong, Zhiqiang Wang, Zhizhi Yu, Dongxiao He, Yuxiao Huang, and Wenjun Wang. 2022. Graph neural network for higher-order dependency networks. In *WWW*. 1622–1630.
- [21] Wei Ju, Zheng Fang, Yiyang Gu, Zequn Liu, Qingqing Long, Ziyue Qiao, Yifang Qin, Jianhao Shen, Fang Sun, Zhiping Xiao, et al. 2024. A comprehensive survey on deep graph representation learning. *Neural Networks* (2024), 106207.
- [22] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv* (2016).
- [23] Kezhi Kong, Jiuhai Chen, John Kirchenbauer, Renkun Ni, C Bayan Bruss, and Tom Goldstein. 2023. GOAT: A global transformer on large-scale graphs. In *ICML*. PMLR, 17375–17390.
- [24] Soo Yong Lee, Fanchen Bu, Jaemin Yoo, and Kijung Shin. 2023. Towards deep attention in graph neural networks: Problems and remedies. In *ICML*. PMLR, 18774–18795.
- [25] Bingheng Li, Erlin Pan, and Zhao Kang. 2024. Pc-conv: Unifying homophily and heterophily with two-fold filtering. In *AAAI*, Vol. 38. 13437–13445.
- [26] Chao Li, Zijie Guo, Kun He, et al. 2024. Long-range meta-path search on large-scale heterogeneous graphs. *NeurIPS* 37 (2024), 44240–44268.
- [27] Guohao Li, Matthias Müller, Bernard Ghanem, and Vladlen Koltun. 2021. Training graph neural networks with 1000 layers. In *ICML*. PMLR, 6437–6449.
- [28] Jianxin Li, Hao Peng, Yuwei Cao, Yingdong Dou, Hekai Zhang, S Yu Philip, and Lifang He. 2021. Higher-order attribute-enhancing heterogeneous graph neural networks. *IEEE TKDE* 35, 1 (2021), 560–574.
- [29] Xiang Li, Chaofan Fu, Zhongying Zhao, Guanjie Zheng, Chao Huang, Yanwei Yu, and Junyu Dong. 2025. Dual-channel multiplex graph neural networks for recommendation. *IEEE TKDE* 37, 6 (2025), 3327–3341.
- [30] Xiang Li, Jianpeng Qi, Zhongying Zhao, Guanjie Zheng, Lei Cao, Junyu Dong, and Yanwei Yu. 2025. UMGAD: Unsupervised Multiplex Graph Anomaly Detection. In *ICDE*. IEEE, 3724–3737.
- [31] Yiming Li, Yanyan Shen, Lei Chen, and Mingxuan Yuan. 2023. Zebra: When temporal graph neural networks meet temporal personalized PageRank. *VLDB* 16, 6 (2023), 1332–1345.
- [32] Yuxuan Liang, Wentao Zhang, Zeang Sheng, Ling Yang, Jiawei Jiang, Yunhai Tong, and Bin Cui. 2024. Hgamlp: Heterogeneous graph attention mlp with de-redundancy mechanism. In *ICDE*. IEEE, 2779–2791.
- [33] Yuxuan Liang, Wentao Zhang, Zeang Sheng, Ling Yang, Quanqing Xu, Jiawei Jiang, Yunhai Tong, and Bin Cui. 2025. Towards scalable and deep graph neural networks via noise masking. In *AAAI*, Vol. 39. 18693–18701.
- [34] Zhuomin Liang, Liang Bai, Xian Yang, and Jiye Liang. 2025. Multi-Channel Disentangled Graph Neural Networks with Different Types of Self-constraints. *IEEE TPAMI* (2025).
- [35] Ningyi Liao, Dingheng Mo, Siqiang Luo, Xiang Li, and Pengcheng Yin. 2022. SCARA: scalable graph neural networks with feature-oriented optimization. *VLDB* 15, 11 (2022), 3240–3248.
- [36] Zhijun Liu, Chao Huang, Yanwei Yu, Baode Fan, and Junyu Dong. 2020. Fast attributed multiplex heterogeneous network embedding. In *CIKM*. 995–1004.
- [37] Qingsong Lv, Ming Ding, Qiang Liu, Yuxiang Chen, Wenzheng Feng, Siming He, Chang Zhou, Jianguo Jiang, Yuxiao Dong, and Jie Tang. 2021. Are we really making much progress? revisiting, benchmarking and refining heterogeneous graph neural networks. In *KDD*.
- [38] Qiheng Mao, Zemin Liu, Chenghao Liu, and Jianling Sun. 2023. Hinormer: Representation learning on heterogeneous information networks with graph transformer. In *WWW*. 599–610.
- [39] Feiping Nie, Yitao Song, Wei Chang, Rong Wang, and Xuelong Li. 2024. Fast Semi-Supervised Learning on Large Graphs: An Improved Green-Function Method. *IEEE TPAMI* (2024).
- [40] Erlin Pan and Zhao Kang. 2023. Beyond homophily: Reconstructing structure for graph-agnostic clustering. In *ICML*. PMLR, 26868–26877.
- [41] Jie Peng, Runlin Lei, and Zhewei Wei. 2024. Beyond over-smoothing: Uncovering the trainability challenges in deep graph neural networks. In *CIKM*. 1878–1887.
- [42] Emanuele Rossi, Fabrizio Frasca, Ben Chamberlain, Davide Eynard, Michael Bronstein, and Federico Monti. 2020. Sign: Scalable inception graph neural networks. *arXiv* (2020).
- [43] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *ESWC*. Springer, 593–607.
- [44] Zhihao Shi, Jie Wang, Zhiwei Zhuang, Xize Liang, Bin Li, and Feng Wu. 2025. Accurate and Scalable Graph Neural Networks via Message Invariance. In *The Thirteenth International Conference on Learning Representations*.
- [45] Changsheng Shui, Xiang Li, Jianpeng Qi, Guiyuan Jiang, and Yanwei Yu. 2024. Hierarchical Graph Contrastive Learning for Review-Enhanced Recommendation. In *ECML-PKDD*. Springer, 423–440.
- [46] Wenxuan Tu, Sihang Zhou, Xinwang Liu, Zhiping Cai, Yawei Zhao, Yue Liu, and Kunlun He. 2025. Wage: Weight-sharing attribute-missing graph autoencoder. *IEEE TPAMI* (2025).
- [47] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *NeurIPS* 30 (2017).
- [48] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv* (2017).
- [49] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. 2019. Heterogeneous graph attention network. In *WWW*. 2022–2032.
- [50] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying graph convolutional networks. In *ICML*. PMLR, 6861–6871.
- [51] Lingfei Wu, Peng Cui, Jian Pei, Liang Zhao, and Xiaojie Guo. 2022. Graph neural networks: foundation, frontiers and applications. In *KDD*. 4840–4841.
- [52] Rui Xue, Haoyu Han, MohamadAli Torkamani, Jian Pei, and Xiaorui Liu. 2023. Lazygcn: Large-scale graph neural networks via lazy propagation. In *ICML*. PMLR, 38926–38937.
- [53] Xiaocheng Yang, Mingyu Yan, Shirui Pan, Xiaochun Ye, and Dongrui Fan. 2023. Simple and efficient heterogeneous graph neural network. In *AAAI*, Vol. 37. 10816–10824.
- [54] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *KDD*. 974–983.
- [55] Lingfan Yu, Jiajun Shen, Jinyang Li, and Adam Lerer. 2020. Scalable graph neural networks for heterogeneous graphs. *arXiv* (2020).
- [56] Pengyang Yu, Chaofan Fu, Yanwei Yu, Chao Huang, Zhongying Zhao, and Junyu Dong. 2022. Multiplex heterogeneous graph convolutional network. In *KDD*.

- [57] Hanqing Zeng, Muhan Zhang, Yinglong Xia, Ajitesh Srivastava, Andrey Malevich, Rajgopal Kannan, Viktor Prasanna, Long Jin, and Ren Chen. 2021. Decoupling the depth and scope of graph neural networks. *NeurIPS* 34 (2021), 19665–19679.
- [58] Zhi Zeng, Minnan Luo, Xiangzheng Kong, Huan Liu, Hao Guo, Hao Yang, Zihan Ma, and Xiang Zhao. 2024. Mitigating World Biases: A Multimodal Multi-View Debiasing Framework for Fake News Video Detection. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 6492–6500.
- [59] Zhi Zeng, Jiaying Wu, Minnan Luo, Xiangzheng Kong, Zihan Ma, Guang Dai, and Qinghua Zheng. 2025. Understand, Refine and Summarize: Multi-View Knowledge Progressive Enhancement Learning for Fake News Video Detection. In *Proceedings of the 33rd ACM International Conference on Multimedia*. 9216–9225.
- [60] Zhi Zeng, Jiaying Wu, Minnan Luo, Herun Wan, Xiangzheng Kong, Zihan Ma, Guang Dai, and Qinghua Zheng. 2025. IMOL: Incomplete-Modality-Tolerant Learning for Multi-Domain Fake News Video Detection. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 30921–30933.
- [61] Chuxu Zhang, Dongjin Song, Chao Huang, Ananthram Swami, and Nitesh V Chawla. 2019. Heterogeneous graph neural network. In *KDD*. 793–803.
- [62] Fanjin Zhang, Xiao Liu, Jie Tang, Yuxiao Dong, Peiran Yao, Jie Zhang, Xiaotao Gu, Yan Wang, Evgeny Kharlamov, Bin Shao, et al. 2022. Oag: Linking entities across large-scale heterogeneous knowledge graphs. *IEEE TKDE* 35, 9 (2022), 9225–9239.
- [63] Heng-Kai Zhang, Yi-Ge Zhang, Zhi Zhou, and Yu-Feng Li. 2024. HONGAT: Graph Attention Networks in the Presence of High-Order Neighbors. In *AAAI*, Vol. 38. 16750–16758.
- [64] Wentao Zhang, Ziqi Yin, Zeang Sheng, Yang Li, Wen Ouyang, Xiaosen Li, Yangyu Tao, Zhi Yang, and Bin Cui. 2022. Graph attention multi-layer perceptron. In *KDD*.
- [65] Ziwei Zhang, Peng Cui, Haoyang Li, Xiao Wang, and Wenwu Zhu. 2018. Billion-scale network embedding with iterative random projection. In *ICDM*. IEEE, 787–796.
- [66] Shuai Zheng, Zhenfeng Zhu, Zhizhe Liu, Youru Li, and Yao Zhao. 2023. Node-oriented spectral filtering for graph neural networks. *IEEE TPAMI* 46, 1 (2023), 388–402.
- [67] Xin Zheng, Miao Zhang, Chunyang Chen, Quoc Viet Hung Nguyen, Xingquan Zhu, and Shirui Pan. 2023. Structure-free graph condensation: From large-scale graphs to condensed graph-free data. *NeurIPS* 36 (2023), 6026–6047.
- [68] Hao Zhu and Piotr Koniusz. 2021. Simple spectral graph convolution. In *ICLR*.
- [69] Shichao Zhu, Chuan Zhou, Shirui Pan, Xingquan Zhu, and Bin Wang. 2019. Relation structure-aware heterogeneous graph neural network. In *ICDM*. IEEE, 1534–1539.

A Supplement

A.1 The Learning Process of ScaleGNN

Algorithm 1 shows the pseudo-code of ScaleGNN framework. The algorithm summarizes the overall process of the proposed ScaleGNN framework. The model first constructs pure k -hop adjacency matrices to isolate neighbor information at different topological scales. It then performs adaptive high-order feature fusion by learning attention weights across multiple hops, and separately aggregates low-order features via the original adjacency. To suppress redundant information introduced by distant neighbors, a local contribution score (LCS) is used to select the most informative nodes, refining the high-order structure. Finally, the node representations are obtained by weighted fusion of the low- and high-order embeddings.

A.2 Time Complexity Analysis

To evaluate the computational efficiency of ScaleGNN, we analyze its time complexity and compare it with existing scalable GNNs. The overall complexity consists of three main components: pre-processing, training, and inference.

Let $|\mathcal{V}|$ be the number of nodes, $|\mathcal{E}|$ be the number of edges, and f be the feature dimension, K be the maximum number of hops, n be the maximum number of top- m_i ($i \in \{1, 2, \dots, K\}$) neighbors. L is the number of layers in MLP classifiers of SIGN, GAMLP, and

Algorithm 1 ScaleGNN

Require: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{X})$, adjacency $\mathbf{A}$, max order K

Ensure: Node representations $\mathbf{H}$

```

1: Construct pure  $i$ -hop adjacency:
2: for  $i = 1$  to  $K$  do
3:    $\mathbf{A}_i = \mathbf{A}^i - \mathbf{A}^{i-1}$ 
4: end for
5: Adaptive high-order feature fusion:
6: Learn weights  $\alpha_1, \dots, \alpha_K$  via softmax,  $\sum_i \alpha_i = 1$ 
7:  $\tilde{\mathbf{A}} = \sum_{i=1}^K \alpha_i \mathbf{A}_i$ 
8:  $\mathbf{H}_{\text{high}} = \sigma(\tilde{\mathbf{A}} \cdot \mathcal{X} \cdot \mathbf{W}_{\text{high}})$ 
9: Low-order feature aggregation:
10:  $\mathbf{H}_{\text{low}} = \sigma(\mathbf{A}^2 \cdot \mathcal{X} \cdot \mathbf{W}_{\text{low}})$ 
11: High-order redundant feature masking:
12: for each node  $v$  do
13:   for  $i = 2$  to  $K$  do
14:     for each neighbor  $u$  in  $\mathbf{A}_i$  do
15:       Calculate  $\text{LCS}(v, u, i)$  by Eq. (6)
16:     end for
17:     Select top- $m_i$  neighbors by LCS, mask others in  $\mathbf{A}_i$ 
18:   end for
19: end for
20: Update  $\tilde{\mathbf{A}}$  and  $\mathbf{H}_{\text{high}}$  using  $\tilde{\mathbf{A}}_{\text{filter}}$ 
21: Feature fusion:
22: Learn  $\beta \in [0, 1]$ 
23:  $\mathbf{H} = \beta \mathbf{H}_{\text{low}} + (1 - \beta) \mathbf{H}_{\text{high}}$ 
24: return  $\mathbf{H}$ 

```

GBP, S in NARS denotes the number of subgraphs, P in SeHGNN denotes the number of divided groups, and R in RpHGNN is the number of node classes. In our proposed ScaleGNN, we perform the following major computations:

- **Multi-hop Adjacency Construction:** To construct order-distilled adjacency matrices $\{\mathbf{A}_i\}_{i=1}^K$ where $\mathbf{A}_i = \mathbf{A}^i - \mathbf{A}^{i-1}$, we perform K sparse matrix multiplications. The cost per multiplication is $O(|\mathcal{E}|)$ in sparse setting, so the total cost is $O(K|\mathcal{E}|)$.
- **LCS-based Neighbor Filtering:** For each node v and k -hop neighbor u , attention-based LCS involves two linear projections ($\mathbf{W}_1 \mathbf{x}_v, \mathbf{W}_2 \mathbf{x}_u$), a dot-product, and softmax computation: (1) Projection cost: amortized $O(|\mathcal{V}|fd_f)$ (precomputed). (2) Attention score computation: $O(K|\mathcal{V}|nd_f)$. (3) The top- m_k filtering: $O(K|\mathcal{V}|n \log n)$ using efficient heap sort. The overall complexity is approximately $O(Kn|\mathcal{V}|f)$ and remains linear in retained neighbors and supports batch-wise parallel execution.
- **Feature Aggregation and Training:** Once filtered adjacency matrices are obtained, the time complexity of aggregation step is $O(Kn|\mathcal{V}|f)$. The time complexity of training and inference is $O(|\mathcal{V}|f^2)$.

Finally the end-to-end time complexity of our ScaleGNN is approximately $O(K|\mathcal{E}| + Kn|\mathcal{V}|f + |\mathcal{V}|f^2)$. Table 4 compares the time complexity of ScaleGNN with other scalable GNN architectures.

Compared to traditional GNNs that suffer from excessive computation due to repeated feature propagation, ScaleGNN optimizes

Table 4: Time complexity comparison of existing deep and scalable GNN models.

Method	Pre-processing	Training	Inference
S ² GC	$O(K \mathcal{E} f)$	$O(\mathcal{V} f^2)$	$O(\mathcal{V} f^2)$
SIGN	$O(K \mathcal{E} f)$	$O(L \mathcal{V} f^2)$	$O(L \mathcal{V} f^2)$
GAMLP	$O(K \mathcal{E} f)$	$O(L \mathcal{V} f^2)$	$O(L \mathcal{V} f^2)$
GBP	$O(K \mathcal{E} f + K\sqrt{ \mathcal{E} } \log \mathcal{V})$	$O(L \mathcal{V} f^2)$	$O(L \mathcal{V} f^2)$
NARS	$O(K \mathcal{V} Sf + K \mathcal{V} fd_f)$	$O(\mathcal{V} f^2)$	$O(\mathcal{V} f^2)$
SeHGNN	$O(K \mathcal{V} Pf + P^2 \mathcal{V} fd_f)$	$O(\mathcal{V} f^2)$	$O(\mathcal{V} f^2)$
RpHGNN	$O(K \mathcal{V} Rf + R \mathcal{V} fd_f)$	$O(\mathcal{V} f^2)$	$O(\mathcal{V} f^2)$
ScaleGNN	$O(Kn \mathcal{V} f) + K \mathcal{E})$	$O(\mathcal{V} f^2)$	$O(\mathcal{V} f^2)$

computational efficiency in two ways: (1) **Sparse high-hop neighbor selection**: By leveraging LCS-based filtering, ScaleGNN significantly reduces the number of unnecessary high-hop neighbors involved in feature propagation, improving computational efficiency. (2) **Lightweight aggregation**: Since ScaleGNN explicitly separates low-hop and high-hop features, it avoids redundant computations, leading to lower memory and runtime costs.

Empirical results on real-world datasets demonstrate that the proposed ScaleGNN achieves faster convergence and lower training time compared to existing model-simplification methods while maintaining high predictive accuracy.

A.3 Datasets

The first three datasets (Citeseer, Cora, and Pubmed) are relatively small and have been adopted by existing work [33], while the latter three (ogbn-arxiv, ogbn-products, and ogbn-papers100M) are large-scale heterogeneous graphs commonly used in scalable HGNN evaluations. The detailed dataset description is shown in Table 1.

A.4 Baselines

We compare our MHCL with the following 12 baselines, which can be deeply divided into four categories:

Traditional GNN methods:

- **R-GCN** [43] introduces relation-specific graph convolution operations to effectively model multi-relational knowledge graphs and achieves strong performance on tasks like link prediction and entity classification.
- **Cluster-GCN** [4] is an efficient GCN algorithm that trains models by sampling dense subgraphs partitioned via graph clustering. It significantly reduces memory and computational overhead, enabling efficient training on million-scale graphs for the first time.
- **HetGNN** [61] is a heterogeneous graph neural network that jointly learns from both heterogeneous graph structure and node content by aggregating multi-type neighbor and attribute information, achieving superior performance on tasks like link prediction and node classification.
- **HAN** [49] employs a hierarchical attention mechanism, combining node-level and semantic-level attention to effectively capture the importance of both meta-path neighbors and meta-paths, enabling expressive node representations in heterogeneous graphs.

- **MAGNN** [10] integrates node content, intermediate nodes along meta-paths, and multiple meta-paths through hierarchical aggregation, enabling more expressive and accurate representations in heterogeneous graphs.
- **Simple-HGN** [37] is a strong and straightforward baseline for heterogeneous graph learning that, with proper input processing, outperforms previous heterogeneous GNN models across standardized benchmark datasets.
- **HINormer** [38] leverages a graph transformer architecture with global-range attention and specialized encoders to effectively capture both structural and heterogeneous relational information, enabling comprehensive node representations in heterogeneous information networks.

Deep GNN methods:

- **SIGN** [42] eliminates the need for neighbor sampling by using efficiently precomputed graph convolutional filters of varying sizes, enabling fast and scalable training and inference on large-scale graphs.
- **S²GC** [68] uses a modified Markov diffusion kernel to efficiently aggregate information from both local and global neighborhoods, capturing multi-scale context while mitigating over-smoothing in deeper graph convolutions.
- **GBP** [2] employs a localized bidirectional propagation strategy to achieve efficient and scalable graph learning, enabling sub-linear time complexity and state-of-the-art performance on massive graphs without relying on sampling.
- **GAMLP** [64] adaptively integrates information from multiple neighborhood scales using attention, effectively addressing over-smoothing and enabling highly scalable and efficient graph learning on large-scale and sparse graphs.
- **GRAND+** [7] enhances scalable graph learning by introducing an efficient pre-computation algorithm for propagation and a confidence-aware consistency loss, enabling fast and accurate semi-supervised learning on large-scale graphs.
- **LazyGNN** [52] captures long-distance dependencies in graphs using shallow models rather than deeper architectures, effectively avoiding neighborhood explosion and achieving efficient, scalable performance on large-scale graphs.

Scalable GNN methods:

- **NARS** [55] efficiently computes neighbor-averaged features over sampled relation subgraphs, enabling scalable and accurate learning on large heterogeneous graphs without complex end-to-end GNN architectures.
- **SeHGNN** [53] streamlines heterogeneous graph learning by using pre-computed mean aggregation and a single-layer structure with long meta-paths, combined with transformer-based semantic fusion, achieving high accuracy and fast training with low complexity.
- **RpHGNN** [17] combines efficient pre-computation with low information loss by using random projection squashing and a relation-wise even-odd propagation scheme, enabling fast and accurate learning on heterogeneous graphs.
- **TOP** [44] eliminates costly out-of-batch message passing by introducing message invariance to convert it into efficient in-batch propagation, enabling fast and scalable GNN training on large graphs with minimal accuracy loss.

Notice that most of the traditional and deep GNN methods encounter out-of-memory (OOM) issues when dealing with large datasets (such as products and paper100M). As for R-GCN, while the original version presents OOM issues with large datasets, it is often used as a baseline for large-scale HGNNs. To adapt R-GCN for large datasets, we follow NARS' experimental setting and adopt the neighbor sampling strategy used by GraphSAGE [14]. Besides, scalable GNN methods, NARS, SeHGNN, and RpHGNN, are all pre-computation-based methods that differ from other end-to-end models. Most of their model runtime is consumed in obtaining the pre-computation tensor for large-scale graphs, with only a tiny amount of time spent on training and inference.

A.5 Implementation Details

We implement all baseline methods according to their provided code. Generally, the size of the hidden embedding d_f is selected from {32, 64, 128, 256, 512}, and the maximum number K of hops is adjustable. For three small datasets, we set the initial epoch number to 100 and select dropout rate, weight decay, and learning rate from {0.1, 0.2, 0.3, 0.4, 0.5, 0.6}, {1e-5, 5e-5, 1e-4, 5e-4, 1e-3, 5e-3, 1e-2} and {0.0001, 0.0005, 0.001, 0.005, 0.01}, respectively. For three large-scale datasets, we set the epoch number to 1000 and the learning rate to 0.001 without weight decay. The embedding dimension d is chosen from {32, 64, 128, 256, 512}, and our method uses GCN as the message aggregator. All experiments are conducted on a machine with an AMD EPYC 7402 24-core CPU and NVIDIA GeForce RTX 4090 (24GB Memory) GPUs.